

講義 28：客製 Table Maintenance 的權限防護與並行控制（授課順序：接在講義 27 之後）

對應練習：ex28 | 答案物件：表 ZTR28_WPARM + 權限物件 ZTR28_WERK + Lock Object EZTR28_WERKS + T-code ZTR28_MAINT + 程式 ZR_TR28_PARAM_MAINT / ZR_TR28_PARAM_LIST

本講重點

- 為什麼 SM30 原生的權限檢查不夠用——業務維度（如「只能改自己工廠」）要自己包一層
- 自訂權限物件（Authorization Object）：ACTVT + 業務欄位（本例 WERKS）
- Lock Object 的鎖定範圍可以比表格主鍵更粗——鎖「一個工廠」而不是「一筆資料」
- VIEW_MAINTENANCE_CALL：從程式呼叫標準 Table Maintenance 的正規做法
- T-code 應該指到「包裝程式」，不是直接指到裸的維護畫面
- Report 主程式的 App bar 按鈕怎麼呼叫另一個 Transaction：SET PF-STATUS + AT USER-COMMAND + CALL TRANSACTION
- PFCG 角色維護：怎麼把自訂權限物件實際指派給一個使用者，讓 AUTHORITY-CHECK 真的能通過

本講六項 GUI-only 步驟（SU21、PFCG 角色維護、SE11 Lock Object、SM30、SE93、SE41），全部都沒有 ADT 可以自動化，逐一附上完整操作路徑，照著做即可，不需要額外查資料。

1. 為什麼要包一層 Authorization Wrapper

講義 21 學過：SM30 產生的維護畫面背後靠 Table Maintenance Generator，存取權限預設是靠 S_TABU_DIS（Table Authorization Group）/ S_TABU_NAM（依表名）這兩個通用權限物件控管——它們回答的問題是「這個使用者能不能維護這張表」，是表格層級的粗粒度控管。

實務上常常需要更細的控管：「王小明只能改工廠 1011 的參數，李小美只能改工廠 2011 的參數」——這種業務維度的權限，SM30 原生機制做不到（除非另外設定進階的 Organizational Criteria，那是更複雜的機制，本課不涉及）。

解法：不要把 T-code 直接指給 SM30，而是自己寫一支「包裝程式（Wrapper）」，T-code 指給這支程式。使用者要維護資料，一定先經過這支程式，程式裡先做：

1. 權限檢查（這個使用者對「這個工廠」有沒有維護權限）
2. 上鎖（避免同一個工廠同時有兩個人在維護，造成第 27 講學過的遺失更新）

兩關都過了，才呼叫標準機制把使用者帶進真正的維護畫面。這是企業導入客製主檔維護時的標準模式，值得完整走一次。

2. SM30 Table Maintenance Generator（跟講義 21 相同手法，欄位換成本題的表）

1. SE11 → 表 ZTR28_WPARM → Utilities → Table Maintenance Generator
2. Authorization Group &NC&（練習用，不檢核）；Function Group 填 ZFG_TR28；Maintenance type 選 One Step

- 產生後可以先直接用 **SM30** 手動測試維護畫面本身 (輸入 View 名稱 **ZTR28_WPARM** → Maintain) , 確認能新增/修改一筆資料——這一步只是確認 View 本身能動, 還沒有套用後面的權限/鎖定 Wrapper, 兩者是獨立的

3. 自訂權限物件: SU21

- 交易碼輸入 **SU21** → Enter
- 左側樹狀選單找一個 Object Class ( **BC** 這個代碼本身不存在, 是分類的字首不是完整代碼, 實測 F4 選單裡查得到的是 **BC_A** (Basis: Administration) 、**BC_C** (Basis - Development Environment) 、**BC_Z** (Basis - Central Functions) 等更細的子分類——本例選 **BC_A**; 實務上依表格所屬模組選對應 Class, 例如 PP 模組相關的表可以選 **PP**) → 對該 Class 按滑鼠右鍵 → **Create** (或工具列的「Create」按鈕)
- Object** 欄位輸入 **ZTR28_WERK** (權限物件名稱上限只有 **10** 碼, **ZTR28_WERKS** 11 碼會超過, 要縮寫成 **ZTR28_WERK**——這點跟 Lock Object / 表格 / Data Element 動輒 16~30 碼的限制不同, 10 碼是權限物件特有的較嚴格限制) , **Text** 填說明 (如「TR28 工廠維護權限」) → Enter
- Authorization Fields** 頁籤, 逐一加兩個欄位 (每個欄位按 **Insert Row** 或直接在空白列輸入) :
 - ACTVT** (標準欄位, Activity, 代表「做什麼」: **01**=Create、**02**=Change、**03**=Display.....SAP 標準的活動代碼清單)
 - WERKS** (本例的業務欄位, 代表「對誰」: 哪個工廠——欄位型別會自動帶出 Data Element **WERKS_D** 的意義, 因為 **WERKS** 是 SAP 標準保留的欄位名稱)
- 存檔 (跳出的 Transport 對話框選 **Local Object**, 練習用途不用建正式傳輸單)
- 工具列 **Activate** (或 Ctrl+F3) ——存檔時如果看到「Permissible activities not maintained for field ACTVT」這種黃色警告 (不是紅色錯誤) 可以先忽略, 那是提醒「還沒設定這個物件允許哪些 **ACTVT** 值」, 不影響本題後續使用; 如果是紅色的「Object class ... does not exist」就要回頭檢查 Class 代碼有沒有打對

ACTVT 幾乎是所有自訂權限物件的標配: 光看「使用者對某張表有沒有權限」不夠, 還要分「只能看」還是「可以改」。標準活動代碼 **01/02/03/06** (刪除) / **08** (顯示變更文件)...可以查 **SU21** 或 **SE11** 顯示 Data Element **ACTVT** 的固定值清單。

 本例的權限物件 **ZTR28_WERK** (無 **S**) 跟 Lock Object **EZTR28_WERKS** (有 **S**, 且 **E** 開頭) 拼法不同, 不要看錯——一開始設計時兩者刻意同名 (只差 **E** 開頭) 容易搞混, 後來因為權限物件的 10 碼上限被迫把其中一個縮寫, 兩者現在拼法不同了, 但因為外觀還是很相似, 操作時務必看清楚字尾有沒有 **S**。

程式裡用 **AUTHORITY-CHECK** 呼叫這個物件:

```
AUTHORITY-CHECK OBJECT 'ZTR28_WERK'
  ID 'ACTVT' FIELD lv_actvt
  ID 'WERKS' FIELD p_werks.
```

```
IF sy-subrc <> 0.
```

" 沒有權限—sy-subrc 常見值 (官方定義) : 4=使用者有這個物件的授權, 但值對不上 (或欄位規格錯誤) ;

" 12=使用者的 User Master Record 裡完全沒有這個物件的任何授權 (不是「物件不存在」, 是「這個人一筆都沒有」,

" 通常代表 PFCG 角色還沒指派給這個人, 或角色沒 Generate/沒 User Comparison)

```
ENDIF.
```

權限物件只是定義了「有哪些欄位可以管控」，真正「誰對什麼值有權限」要靠角色維護 (PFCG) ——建好權限物件之後，還沒有任何人真的擁有這個權限，AUTHORITY-CHECK 一律會失敗，直到走完下面第 2.1 節的 PFCG 流程為止。

3.1 PFCG 角色維護：把權限物件真正指派給使用者

這一步是整套機制真正生效的關鍵，很多課程或文件會跳過，但沒有這一步，前面 SU21 建的物件形同虛設。完整走一次：

1. 交易碼輸入 **PFCG** → Enter
2. **Role** 欄位輸入角色名稱 (如 **ZTR28_MAINT_ROLE**，Z 開頭) → **Single Role** 按鈕 (不是 Composite Role)
3. **Description** 頁籤：Description 欄位填角色說明 (如「TR28 工廠參數維護角色」) → 存檔
4. **Menu** 頁籤 (讓角色使用者可以直接從選單找到這支交易，非必要但建議做)：
 - 工具列 **Transaction** 按鈕 (或右鍵 → Insert Transaction)
 - 輸入 T-code **ZTR28_MAINT** → Enter，選單樹會出現這一項
5. **Authorizations** 頁籤 → 按鉛筆圖示 **Change Authorization Data**：
 - 系統會自動帶入 T-code 本身需要的標準權限物件 (如 **S_TCODE**，因為 Menu 頁籤加了 **ZTR28_MAINT**)，但不會自動帶入我們自訂的 **ZTR28_WERK**——自訂物件沒有跟 T-code 自動關聯，要手動加
 - 工具列 **Manually** (手動) 按鈕 → 輸入 **ZTR28_WERK** → Enter，這個物件的節點會出現在權限樹狀清單裡
 - 展開 **ZTR28_WERK** 節點，把 **ACTVT** 欄位的值改成 **02** (Change；如果也想順便給顯示權限可以再加一行 **03**)
 - **WERKS** 欄位填 **1011** (想授權的工廠代碼；教學上建議先只給 **1011**，具體體會「只能改自己工廠」的效果——真的要開放全部工廠可以填 *)
 - 檢查 **S_TCODE** 節點底下的 **TCD** 欄位確實已經有 **ZTR28_MAINT**
6. 工具列齒輪圖示 **Generate** (產生 Profile) → 跳出的 Profile 名稱視窗直接 Enter 接受系統預設值
7. 回到角色主畫面 (可能要按左上角綠色返回箭頭) → **User** 頁籤 → **User ID** 欄位輸入自己的使用者代號 → Enter
8. 工具列 **User Comparison** (使用者比對，通常是兩個人形疊在一起的圖示，或選單 Utilities → User Comparison) → 跳出畫面按 **Complete Comparison**
9. 存檔

驗證方式：登出重新登入 (或另開一個新 Session/Mode)，執行 T-code **ZTR28_MAINT** (**p_werks=1011**)，應該看到「權限檢查通過」而不是先前的「權限不足」。如果還是失敗，交易碼 **SU53** (顯示上一次權限失敗的畫面) 是排查權限問題最直接的工具，可以直接看到是哪個物件、哪個欄位值沒過。

4. Lock Object 的鎖定範圍可以比主鍵更粗

講義 27 的 **EZTR21_STUD** 鎖 **MANDT+ID** (表格完整主鍵)，意思是「鎖一筆學生資料」。本題的表 **ZTR28_WPARM** 主鍵是 **MANDT+WERKS+PARAM** (三個欄位)，但 Lock Object **EZTR28_WERKS** 的 Lock Parameters 只勾 **MANDT+WERKS**——刻意不勾 **PARAM**。

為什麼：業務需求是「同一個工廠不能有兩人同時維護」，不是「同一個參數列不能有兩人同時改」——如果鎖到 **PARAM** 這麼細，兩個人分別改同工廠的不同參數列會被允許同時進行，但兩人事實上都在同一個 SM30 畫面 (同一個工廠的整批參數列表) 裡操作，容易互相覆蓋彼此看到的畫面快照。鎖定範圍要對應到「使用者實際上在爭搶的資源邊界」，不是機械式地照抄表格主鍵——這是 Lock Object 設計上最容易被忽略的一個判斷。

SE11 建立步驟 (跟講義 27 相同流程，範圍不同)：

1. 交易碼輸入 **SE11** → 左側選 **Lock Object** → 輸入 **EZTR28_WERKS** (系統強制規定要 **E** 開頭，不是單純的命名慣例——打別的字首存檔會直接跳出警告「Start the lock object names with the prefix 'E'」。⚠ 這裡容易跟第 3 節建的權限物件 **ZTR28_WERK** 搞混，兩者外觀相似但拼法不同 (**ZTR28_WERK** 無 **S** / **EZTR28_WERKS** 有 **S**)，輸入時務必看清楚) → **Create**
2. **Tables** 頁籤：Primary Table 填 **ZTR28_WPARM** → Enter (系統會自動帶出這張表的完整欄位清單)
3. **Lock Parameters** 頁籤：勾選 **MANDT**、**WERKS** 兩個欄位的 **Lock parameter** 核取方塊——**PARAM** 欄位保持不勾
4. **Lock Mode** 欄位選 **E** (Exclusive/Write Lock)
5. 存檔 (Local Object)、工具列 **Activate**
6. 查詢系統自動產生的 **FM**：該畫面 → Utilities → Generated Objects，應該看到 **ENQUEUE_EZTR28_WERKS** / **DEQUEUE_EZTR28_WERKS**；也可以 SE37 直接 Display 這兩個 FM，確認 Import 參數只有 **MANDT** / **WERKS** (沒有 **PARAM**，證實鎖定範圍確實只到工廠層級)

5. VIEW_MAINTENANCE_CALL：從程式呼叫標準 Table Maintenance

土法煉鋼的做法是在程式裡 **CALL TRANSACTION 'SM30'**，但這樣沒辦法乾淨地指定要維護哪個 View、也繞不過 SM30 自己的初始畫面。SAP 提供了正規的函式模組：

```
CALL FUNCTION 'VIEW_MAINTENANCE_CALL'
  EXPORTING
    action      = 'U'          " U=維護 ( Update )、S=只顯示 ( Show )、T=帶Transport單維護
    view_name   = 'ZTR28_WPARM'
  EXCEPTIONS
    client_reference      = 1
    foreign_lock          = 2
    invalid_action        = 3
    no_clientindependent_auth = 4
    system_failure        = 5
    OTHERS                = 6.
```

這支 FM 內部會再做一次它自己的權限檢查 (**S_TABU_DIS/S_TABU_NAM**) 跟再上一次它自己的鎖 (**VIEW_ENQUEUE**，鎖的是整個 View，不是我們自訂的 Lock Object)——這是完全獨立、疊加在我們自訂 Wrapper 之上的第二層保護，兩層互不衝突：我們的 Wrapper 負責業務維度 (工廠)，**VIEW_MAINTENANCE_CALL** 自己負責表格層級的通用維護權限與鎖定。

action = 'S' (只顯示) 搭配 **ACTVT = '03'** 的權限檢查，可以做出一個「只能看、不能改」的模式，這正是本題 Wrapper 程式 **p_disp** 核取方塊要做的事。

5.1 用 **dba_sellist** 篩選：只顯示使用者被授權的那個工廠

只做完 **AUTHORITY-CHECK** 還不夠——它只回答「這個人能不能對工廠 **1011** 做維護」，不會自動限制他進了 **VIEW_MAINTENANCE_CALL** 畫面之後只看得到 **1011** 的資料。如果不額外處理，一個只被授權 **WERKS=1011** 的使用者，一樣可以在維護畫面裡看到、甚至改到其他工廠的列——這是很容易漏掉的一步。

`VIEW_MAINTENANCE_CALL` 有一個 `TABLES dba_sellist` 參數 (型別 `VIMSELLIST`) , 可以帶入篩選條件 , 效果類似幫這次維護動作先套一個 `WHERE` 子句 :

```
DATA: lt_sellist TYPE STANDARD TABLE OF vimsellist,
      ls_sellist TYPE vimsellist.

CLEAR ls_sellist.
ls_sellist-viewfield = 'WERKS'.    " 要篩選的欄位名稱 ( 大寫 )
ls_sellist-operator  = 'EQ'.      " 比較運算子 : EQ/NE/GT/GE/LT/LE/LK ( Like )
ls_sellist-value     = p_werks.   " 要比對的值
ls_sellist-tabix     = 1.         " 條件的序號 ( 多筆條件用 and_or 欄位串接 AND/OR )
APPEND ls_sellist TO lt_sellist.

CALL FUNCTION 'VIEW_MAINTENANCE_CALL'
  EXPORTING
    action      = 'U'
    view_name   = 'ZTR28_WPARAM'
  TABLES
    dba_sellist = lt_sellist
  EXCEPTIONS
    ...
```

帶了這個篩選之後 , 維護畫面一開啟就只會列出 `WERKS = p_werks` 這個工廠的資料列 , 使用者不會不小心看到或改到別的工程——這一步刻意跟下一節 `ZR_TR28_PARAM_LIST` 那顆「直接呼叫 **SM30**」的按鈕做對照 : 那顆按鈕完全沒有篩選 , 會看到整張表所有工廠的資料 , 兩者的差異正好示範了「隨便呼叫 **SM30**」跟「透過 **Wrapper** 正規呼叫」在資料曝露範圍上的落差。

⚠ `VIEW_MAINTENANCE_CALL` 會真的開出一個完整畫面 , 等使用者操作完才會返回——這代表它沒辦法用 ADT 的無頭 `programrun` API 驗證 (`headless` 呼叫會卡在等畫面回應 , 最終連線逾時斷線 `RFC_CLOSED`) , 只能請使用者在真實 SAP GUI 測試 , 跟前面題目遇過的 `ALV / Smartform` 是同一類限制。還有一個連帶影響 : 如果 `Wrapper` 程式在呼叫這支 FM「之前」已經成功 `ENQUEUE_EZTR28_WERKS` , `programrun` 卡在這裡逾時斷線時 , 程式不會執行到後面的 `DEQUEUE_EZTR28_WERKS` (因為程式根本沒跑完) , 理論上鎖會殘留——**SM12** 跟 **Lock Object** 的關係、以及這種殘留鎖該怎麼排查與清除 , 見講義 27 第 7 節 , 那一節已經用這個真實情境當例子寫進去了。

6. T-code 要指給 `Wrapper` , 不是指給裸的 **SM30**

1. 交易碼輸入 **SE93** → **Transaction Code** 欄位輸入 `ZTR28_MAINT` → **Create**
2. **Short Text** 填說明 (如「**TR28** 工廠參數維護」)
3. 型態選 **Program and selection screen (Report transaction)** (單選按鈕) → **Enter**
4. **Program** 欄位填 `ZR_TR28_PARAM_MAINT` (我們的 `Wrapper` 程式 , 不是 **SM30**、也不是直接填 **View** 名稱) , **Selection Screen** 欄位填 `1000` (一般報表選取畫面的慣例編號 , 見 **Enhancement** 課程 `en02` 的說明)
5. 存檔 (**Local Object**)

這是整個模式能不能生效的關鍵一步：如果貪方便，直接開一個 T-code 指給 SM30、或是把 `S_TABU_DIS/S_TABU_NAM` 開放給所有人，使用者就能繞過我們寫的 Wrapper 直接進 SM30 維護，前面做的權限檢查與 Lock Object 就完全形同虛設。實務上要搭配：一般使用者的角色不給 **SM30 / 裸 View** 維護的權限，只給這支 Wrapper 的 **T-code**，才能確保大家只能走這條有檢查的路。

7. 選取畫面上的按鈕：SELECTION-SCREEN FUNCTION KEY

`ZR_TR28_PARAM_LIST` 是「主檔的瀏覽入口」——它的選取畫面（輸入 `s_werks` 篩選條件的那個畫面）上直接加一顆按鈕，讓使用者不用先跑出清單、也不用另外去記 T-code，就能捷徑跳去維護主檔。這跟 ALV 工具列按鈕（講義 9 進階篇，`IT_EVENTS`）、或前面版本一度用過的清單畫面 `SET PF-STATUS/AT USER-COMMAND`（那是掛在執行完畢後的輸出清單上）都不一樣——這裡要按的是選取畫面本身的按鈕，語法是 `SELECTION-SCREEN FUNCTION KEY`：

1. 宣告 **TABLES: sscrfields.**——這是系統結構，專門用來承接選取畫面上的功能鍵被按下時的資訊（哪一個按鈕、代碼是什麼）
2. 宣告功能鍵：**SELECTION-SCREEN FUNCTION KEY 1.**（位置很重要，這一行要放在選取畫面相關宣告區塊，即 `PARAMETERS/SELECT-OPTIONS` 附近，不能放在事件區塊裡）——最多可以宣告到 **FUNCTION KEY 4**（四顆自訂按鈕），系統固定把它們的 Function Code 命名為 `'FC01' / 'FC02' / 'FC03' / 'FC04'`，這四個代碼是系統保留的，不能自己改
3. 在 **INITIALIZATION** 事件裡設定按鈕文字：`sscrfields-functxt_01 = '維護主檔(SM30)'`。（`functxt_01` 對應 **FUNCTION KEY 1**；有幾個按鈕就對應填 `functxt_01 ~ functxt_04`）
4. 在 **AT SELECTION-SCREEN** 事件裡判斷按鈕被按下、處理動作：

```
TABLES: ztr28_wparm, sscrfields.

SELECT-OPTIONS s_werks FOR ztr28_wparm-werks.

SELECTION-SCREEN FUNCTION KEY 1.

INITIALIZATION.
    sscrfields-functxt_01 = '維護主檔(SM30)'.

AT SELECTION-SCREEN.
    CASE sscrfields-ucomm.
        WHEN 'FC01'.
            SET PARAMETER ID 'VIM' FIELD 'ZTR28_WPARM'.
            CALL TRANSACTION 'SM30' AND SKIP FIRST SCREEN.
    ENDCASE.
```

`SET PARAMETER ID 'VIM'` 是把要維護的 View 名稱先塞進 SPA/GPA 記憶體（`'VIM'` 是 SM30 View 名稱欄位對應的標準 Parameter ID），接著 `CALL TRANSACTION 'SM30' AND SKIP FIRST SCREEN` 才能直接跳過 SM30 自己的初始輸入畫面，一步到位進到 `ZTR28_WPARM` 的維護畫面。

⚠ 這顆按鈕故意示範「沒有保護」的做法，跟 `ZR_TR28_PARAM_MAINT` 對照

這裡刻意不透過 `ZR_TR28_PARAM_MAINT` 那個 Wrapper，而是直接 `CALL TRANSACTION 'SM30'`——這是為了跟 Wrapper 的正規做法做對比教學，兩者差在：

	ZR_TR28_PARAM_LIST 的按鈕	ZR_TR28_PARAM_MAINT (T-code ZTR28_MAINT)
呼叫方式	直接 CALL TRANSACTION 'SM30'	走完整的 Wrapper 流程
業務權限檢查 (ZTR28_WERK)	✗ 沒有	✓ 有
Lock Object (EZTR28_WERKS)	✗ 沒有	✓ 有
資料篩選 (只看單一工廠)	✗ 沒有，看得到整張表所有工廠	✓ 有 (見上一節 dba_sellist)
適合場景	純示範對照用；實務上若真的要開放，只能給完全信任、本來就有 SM30/S_TABU_DIS 權限的維運人員	一般使用者的正式入口

這正是本題最重要的教學重點：同樣是「從另一支程式跳進主檔維護」，寫法上的差別（有沒有包一層 Wrapper）決定了資安上是天壤之別。實務上兩個入口不會同時開放給一般使用者——會像講義第 6 節說的，只把 T-code **ZTR28_MAINT** 的權限給一般使用者，**ZR_TR28_PARAM_LIST** 這顆按鈕的存在本身就是要提醒學員「這樣寫是不對的」，不是真的建議這樣上線。

8. 整體流程總結



四層防護（T-code 權限、業務權限、Lock Object、SM30 原生防護）疊在一起，缺任何一層都可能被繞過——這正是企業客製常見「看似簡單的維護畫面，其實包了好幾層防護」的真實樣貌。